

统计学概论

杨文志
安徽大学大数据与统计学院
wzyang@ahu.edu.cn

第10章 模型选择与正则化

- 模型选择问题提出
- 子集选择法
- 基于压缩估计的变量选择
- 基于压缩估计的组变量选择
- R语言实现

10.1 模型选择问题提出

模型选择，是指利用统计方法，从众多自变量中，选择显著的、最能解释因变量变化的那一部分自变量参与建模。在统计建模中，模型选择是重要的也是核心的步骤之一。

子集选择法

正则化方法

降维法

例10.1 银行评估信用风险

- 银行在审核客户的信用卡申请资料时，由于填入的信息繁多而全面，若建立一个包含所有指标的Logistic模型预测客户是否有违约风险，那么**模型的自变量维度将很大，待估参数非常多**。
- 而且这其中不乏对分类影响十分微小的变量，将其加入模型并不会带来显著的预测效果。此外，指标之间**共线性问题**也常常存在。
- 对于这些高度相关的指标，是否只需选择其中的代表进入模型即可？如何更加有效地解决这些问题？

10.2.1 子集选择法

最优子集法运用了**穷举**的思想，对 p 个预测变量的所有可能组合逐一进行拟合，找到最优的模型。算法如下：

1. 记不含预测变量的零模型为 M_0 。
2. 对 $k = 1, 2, \dots, p$:
 - (a) 拟合 C_p^k 个包含 k 个预测变量的模型；
 - (b) 在上述 C_p^k 个模型中，选择残差平方和RSS最小或 R^2 最大的模型作为最优模型，记为 M_k 。
3. 对于得到的 $p + 1$ 个模型 $M_0, M_1, \dots, M_p$ ，进一步根据交叉验证法、 C_p 、AIC、BIC或调整的 R^2 选出一个最优模型，即为最后得到的最优模型。

- 最优子集法的**计算效率较低**，不适用于维数很大的情况。
- 最优子集法得到的模型虽然能很好地拟合训练数据，但往往对新数据的预测效果并不理想，即会存在**过拟合**和**系数估计方差高**的问题。

10.2.2 逐步选择法

1. 向前逐步选择

向前逐步选择法是以一个不包含任何预测变量的零模型为起点，依次往模型中添加变量，每次只将能够最大限度地提升模型效果的变量加入模型中，直到所有的预测变量都包含在模型中。

1. 记不含预测变量的零模型为 M_0 。
2. 对 $k = 1, 2, \dots, p - 1$:
 - (1) 拟合 $p - k$ 个模型，每个模型都是在 M_k 的基础上只增加一个变量；
 - (2) 在上述 $p - k$ 个模型中，选择残差平方和RSS最小或 R^2 最大的模型作为最优模型，记为 M_{k+1} 。
3. 对于得到的 $p + 1$ 个模型 $M_0, M_1, \dots, M_p$ ，进一步根据交叉验证法、 C_p 、AIC、BIC或调整的 R^2 选出一个最优模型，即为最后得到的最优模型。

- 向前逐步选择法需要拟合的模型个数为 $\sum_{k=0}^{p-1} (p - k) = 1 + p(p + 1)/2$ ，所以当 p 较大时，它与最优子集法相比较，在运算效率上具有很大的优势。
- 不过，向前逐步选择法存在的一个问题是，它无法保证找到的模型是 2^p 个模型中最优的。

变量数	向前逐步法	最优子集法
1	X_1	X_1
2	X_1, X_2	X_2, X_3

10.2.2 逐步选择法

2. 向后逐步选择

向后逐步选择法从含有所有变量的模型开始，依次剔除不显著的变量。

1. 记包含全部 p 个预测变量的全模型为 M_p 。
2. 对 $k = p, p - 1, \dots, 1$:
 - (a) 拟合 k 个模型，每个模型都是在 M_k 的基础上只减少一个变量；
 - (b) 在上述 k 个模型中，选择残差平方和RSS最小或 R^2 最大的模型作为最优模型，记为 M_{k-1} 。
3. 对于得到的 $p + 1$ 个模型 $M_0, M_1, \dots, M_p$ ，进一步根据交叉验证法、 C_p 、AIC、BIC或调整的 R^2 选出一个最优模型，即为最后得到的最优模型。

- 与向前逐步选择法类似，向后逐步选择法需要拟合的模型个数同样为 $\sum_{k=0}^{p-1} (p - k) = 1 + p(p + 1)/2$ ，大大少于最优子集法
- 向后逐步选择法也无法保证找到的模型是 2^p 个模型中最优的。
- 向后选择方法还需要满足 $n > p$ 的条件，这样才能保证全模型是可以拟合的，而向前逐步选择法则不需要这个条件限制。因此，当 p 非常大时，应选择向前逐步选择法。

3. 向前向后选择法

还有一种将向前逐步选择法和向后逐步选择法相结合的方法。该方法也是逐次将变量加入模型中，不同的是，在引入新变量的同时，也剔除不能提升模型拟合效果的变量。这种方法在试图达到最优子集法效果的同时也保留了向前和向后逐步选择法在计算上的优势。

10.2.3 模型选择准则

- 在最优子集算法、向前和向后选择算法中，步骤2和3都需要选择最优模型，其中传统的残差平方和 RSS 和 R^2 可以用于步骤2中对具有相同变量个数的模型进行选择，但不适用于步骤3中对变量个数不同的模型进行选择。
- 这是由于随着模型中变量数的增加，残差平方和 RSS 会不断减小， R^2 会不断增大，所以包含所有预测变量的模型总能具有最小的 RSS 和最大的 R^2 ，因为它们只与训练误差有关。
- 而实际中，我们希望找到的测试误差小的模型，即**泛化能力好的模型**。一种方法是对于训练误差进行适当调整，间接估计测试误差，比如 C_p 、AIC、BIC和调整的 R^2 ；
- 另一种方法就是**交叉验证法**，直接估计测试误差。

10.2.3 模型选择准则

1. C_p 准则

若采用**最小二乘法**拟合一个包含 d 个预测变量的模型，则 C_p 的值为：

$$C_p = \frac{1}{n}(RSS + d\hat{\sigma}^2)$$

其中， RSS 为残差平方和， $\hat{\sigma}^2$ 是各个因变量观测误差 ϵ 的方差的估计值。通常而言，测试误差较低的模型其 C_p 取值也更小，**因而在模型选择时，应选择 C_p 取值最小的模型**。

2. AIC准则

AIC准则适用于许多使用**极大似然法**进行拟合的模型，它的一般公式为：

$$AIC = -2\log L(\hat{\theta}) + 2d$$

其中，等号右边第一项为负对数似然函数，第二项是对模型参数个数（**模型复杂度**）的惩罚。实际应用中，我们也应选取**AIC取值最小的模型**。

10.2.3 模型选择准则

2. AIC 准则

对于标准线性回归模型而言，极大似然估计和最小二乘估计是等价的，此时，模型的AIC值为：

$$AIC = \frac{1}{n\hat{\sigma}^2} (RSS + 2d\hat{\sigma}^2)$$

可以看出，对于最小二乘法而言， C_p 和AIC是成比例的。

3. BIC 准则

BIC 准则是从贝叶斯的角度推导出来的，与AIC 准则相似，都是用于最大化似然函数的拟合。对于包含 d 个预测变量的模型，BIC的一般公式为：

$$BIC = -2\log L(\hat{\theta}) + d\log n$$

可以看出BIC与AIC非常相似，只是把AIC中的2换成了 $\log n$ 。所以，当 $n > e^2$ 时，BIC对复杂模型的惩罚更大，故更倾向于选取简单的模型。同样类似于 C_p ，测试误差较低的模型BIC的取值也较低，故通常选择具有最低BIC值的模型最为最优模型，并且对于最小二乘估计，BIC可写成：

$$BIC = \frac{1}{n} (RSS + \log(n)d\hat{\sigma}^2)$$

10.2.3 模型选择准则

4. 调整的 R^2

对于包含 d 个变量的最小二乘估计，其调整的 R^2 可由下式计算得到：

$$\text{调整的} R^2 = 1 - \frac{RSS/(n-d-1)}{TSS/(n-1)}$$

其中 $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ 为残差平方和， $TSS = \sum_{i=1}^n (y_i - \bar{y})^2$ 是因变量的总平方和。与前面介绍的三个准则不同，调整的 R^2 越大，模型的测试误差越低。这里指出，随着模型包含的变量个数 d 的增大， RSS 逐渐减小，不过 $RSS/(n-d-1)$ 可能增大也可能减小，故不存在随着模型包含的变量个数越多，调整的 R^2 就越大的问题。

10.3 基于压缩估计的变量选择

基于压缩估计的变量选择

传统的子集选择法虽然思想十分简单，但是存在许多缺陷。

首先，子集选择法是一个离散而不稳定的过程，模型选择会因数据集的微小变化而变化；

其次，模型选择和参数估计分两步进行，后续的参数估计没有考虑模型选择产生的偏误，从而会低估实际方差；

最后，子集选择法的计算量相对比较大。

一种改进方法就是**基于惩罚函数 (penalty function) 的压缩估计法**，其思想是通过惩罚函数约束模型的回归系数，同步实现变量选择和系数估计，模型估计是一个连续的过程，因而稳健性高。

惩罚函数法根据变量间的结构，主要分为三类，分别是**逐个变量选择** (individual variable selection)，**整组变量选择** (group variable selection) 和**双层变量选择** (bi-level variable selection)。

10.3 基于压缩估计的变量选择

惩罚函数法和最小二乘法或者最大似然估计类似，也是求解优化问题。假设自变量为 $X \in R^p$ ，因变量记为 Y ，回归系数记为 β ，截距项为 β_0 。目标函数的一般形式为

$$\min_{\beta} Q(\beta_0, \beta) = \min_{\beta} \{L(\beta_0, \beta|Y, X) + P(|\beta|; \lambda)\} \quad (11.1)$$

其中 $L(\beta_0, \beta|Y, X)$ 是损失函数，不同模型的损失函数形式不同，通常有最小二乘函数、似然函数的负向变换。例如，线性回归模型的损失函数常用最小二乘函数，即

$$L(\beta_0, \beta|Y, X) = (Y - X\beta - \beta_0)^T (Y - X\beta - \beta_0) / n \quad (11.2)$$

而对于Logistic回归，它的损失函数为极大似然函数的负向变换，比如

$$L(\beta_0, \beta|Y, X) = -l(\beta_0, \beta) / n = \left(-(\beta_0 + X\beta)^T Y + \log(1 + e^{(\beta_0 + X\beta)^T 1_n}) \right) / n \quad (11.3)$$

10.3.1 LASSO惩罚

惩罚函数方法中具有里程碑意义的是由Tibshirani 1996年提出的Lasso (Least Absolute Shrinkage and Selection Operator) 方法。LASSO惩罚函数为

$$P_{Lasso}(|\beta|; \lambda) = \lambda \sum_{i=1}^p |\beta_i| \quad (11.4)$$

对于LASSO惩罚问题，式 (11.1) 等价于如下带约束的优化问题：

$$\begin{aligned} \hat{\beta} &= \arg \min_{\beta} \{L(\beta_0, \beta|Y, X)\} \\ s.t. & \sum_{i=1}^p |\beta_i| \leq t \end{aligned} \quad (11.5)$$

式 (11.4) 中的参数 λ 和式 (11.5) 中的参数 t 存在一一对应的关系。要特别指出的是，**惩罚函数一般不对截距项进行压缩。**

10.3.1 LASSO惩罚

为了便于理解Lasso方法的原理，我们首先介绍一下**岭回归 (Ridge regression)**，它是Hoerl和Kennard于1970年提出的，其惩罚函数为

$$P_{Ridge}(|\beta|; \lambda) = \lambda \sum_{i=1}^p \beta_i^2 \quad (11.6)$$

类似式 (11.5) 可转化为约束条件： $\sum_{i=1}^p \beta_i^2 \leq t$

Lasso虽然和Ridge形式上类似，但是性质上有着重要的差别。由于Lasso惩罚函数在**零点不可导**，因此当大到一定程度时，**可将部分系数压缩为零**，这样连续地实现变量选择。而**Ridge是关于连续可导的，无法进行变量选择。**

调整参数 λ 除了平衡参数的压缩程度和损失函数外，另一个作用是权重的作用。

10.3.2 SCAD惩罚

- 在LASSO惩罚函数中，对系数的压缩权重都是 λ 。然而这存在一定的弊端，对**所有参数一视同仁**，会导致大的系数被过分压缩，带来较大的估计偏差。
- 理想的状态是大系数和小系数要区别对待，大系数不应该被压缩，以降低它们估计的偏差，而小系数尤其是接近0的系数要重点压缩，以更好地识别到不显著变量。

Fan和Li(2001)提出SCAD惩罚，其惩罚函数如下：

$$P_{SCAD}(|\beta|; \lambda, a) = \begin{cases} \lambda |\beta| & \text{若 } 0 \leq |\beta| < \lambda \\ -\frac{\beta^2 - 2a\lambda|\beta| + \lambda^2}{2(a-1)} & \text{若 } \lambda \leq |\beta| < a\lambda \\ (a+1)\lambda^2 / 2 & \text{其它} \end{cases} \quad (11.7)$$

10.3.3 MCP惩罚

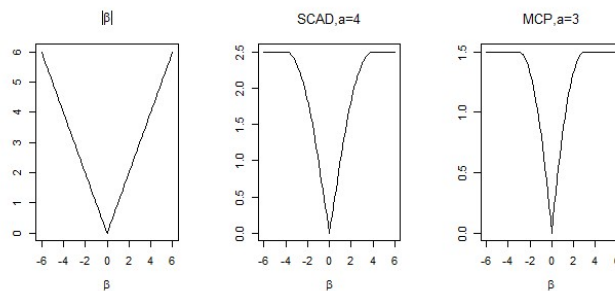
Zhang (2010) 提出的**MCP** (Minimax Concave Penalty) 是与SCAD类似的方法，也是理论性质良好的惩罚方法。它对 $|\beta|$ 也是分阶段惩罚，避免大系数过度被压缩，惩罚函数为

$$P_{MCP}(|\beta|; \lambda, a) = \begin{cases} \lambda |\beta| - \frac{|\beta|^2}{2a}, & \text{若 } |\beta| \leq a\lambda \\ \frac{a\lambda^2}{2}, & \text{若 } |\beta| > a\lambda \end{cases} \quad P'_{MCP}(|\beta|; \lambda, a) = \begin{cases} \lambda - \frac{|\beta|}{a} & \text{若 } |\beta| \leq a\lambda \\ 0 & \text{若 } |\beta| > a\lambda \end{cases}$$

其中参数 $a > 0$ 为待定参数。可以看出在 $|\beta| < a\lambda$ 时，MCP函数的一阶导数随 $|\beta|$ 增大而减小，即 $|\beta|$ 越大，惩罚函数上升越缓慢；当 $|\beta| > a\lambda$ 时，惩罚函数的一阶导数为0，即对大的回归系数不惩罚。MCP也同样改善了Lasso过度惩罚大系数的缺点。

10.3.4 MCP惩罚

下面，从图形上来理解Lasso，SCAD，MCP的压缩特征。



在参数值 β 较小时，SCAD、MCP、Lasso都是 β 的线性函数：

随着 β 的增大，SCAD、MCP由直线惩罚变为曲线惩罚，惩罚力度会慢慢减弱，但是Lasso一直是线性惩罚。
当参数大到一定程度时，SCAD和MCP的值与 β 无关，即不再对 β 进行惩罚，这样可以保证大系数被选入模型。

10.3.5 调整参数选择

调整参数 λ 选择常用的方法有交叉验证（Cross Validation, CV）、广义交叉验证（GCV）、广义信息准则（GIC）、AIC、BIC、风险膨胀准则（RIC）、 C_p 准则等

其中，用CV求解调整参数的思路是：

- (1) 随机地将样本分为等量的 k 份；
- (2) 选择其中 $k - 1$ 份样本作为训练集（train set），用来构建模型，剩下的样本作为测试集（test set）用于检验模型的预测能力，得到这份样本的预测误差平方和；
- (3) 循环第（2）步直到所有样本都被预测一次且仅一次，加总所有样本的预测误差平方和（PRESS）；
- (4) 对于每个可能取值的 λ ，计算它们相应的PRESS值，选择PRESS最小时的 λ 作为最优调整参数估计值。

10.4 基于压缩的组变量选择

组变量选择的含义是，某些变量会因为特殊关系而“绑”在一起**作为一个整体参与变量选择**的过程，绑在一起的变量被选择的结果是相同的，要么全部选入，要么全部剔除。既然多个变量作为整体才能进行组变量选择，那么**确定变量能否视为一个整体**是首要解决的问题，或者说满足什么前提下这种整体性才成立。

在某些实际应用中，自变量呈现分组结构，最常见的是**先验信息形成的自然分组结构**。合理的变量选择方法必须能够正确地剔除整组变量而非单个变量。

这类方法的惩罚目标函数与式 (11.1) 有所不同，它是对参数以组为整体进行惩罚，一般形式如下：

$$\min_{\beta} Q(\beta_0, \beta) = \min_{\beta} \left\{ L(\beta_0, \beta | Y, X) + \sum_{j=1}^J P(|\beta^{(j)}|; \lambda) \right\}$$

其中，回归系数向量分为 J 个组，即 $\beta^T = (\beta^{(1)T}, \beta^{(2)T}, \dots, \beta^{(J)T})$ 。 $X_j = (X_{j1}, X_{j2}, \dots, X_{jp_j})$ 和 $\beta^{(j)} = (\beta_1^{(j)}, \beta_2^{(j)}, \dots, \beta_{p_j}^{(j)})^T$ 分别是第 j 组自变量的 $n \times p_j$ 维设计矩阵和 $p_j \times 1$ 维回归系数， $\sum_{j=1}^J p_j = p$ 。

10.4 基于压缩的组变量选择

接下来介绍组变量选择方法，最常用的是Yuan和Lin (2006) 提出的Group LASSO。它的惩罚函数为

$$P_{GLasso}(|\beta|; \lambda) = \lambda \sum_{j=1}^J \sqrt{p_j} \|\beta^{(j)}\|_2$$

其中 $\|\cdot\|_2$ 为 L_2 范数。从该方法的惩罚函数形式上可知，同组变量的回归系数 $\beta^{(j)} = (\beta_1^{(j)}, \beta_2^{(j)}, \dots, \beta_{p_j}^{(j)})^T$ 是以 $(\beta_1^{(j)})^2 + \dots + (\beta_{p_j}^{(j)})^2$ 的关系存在，可理解为**组内惩罚是岭惩罚**，由第二节可知岭惩罚不能选择变量，因此这里Group Lasso在组内也是不能选择变量的。组间系数 $\beta^T = (\beta^{(1)T}, \beta^{(2)T}, \dots, \beta^{(J)T})$ 的构成形式为 $(P_{Ridge}(\beta^{(1)}))^{1/2} + \dots + (P_{Ridge}(\beta^{(J)}))^{1/2}$ ，其中函数 P_{Ridge} 是岭惩罚函数的形式，那么组间是具有变量选择功能的**Bridge惩罚** (Frank IE 和Friedman JH在1993年提出)。

10.4 基于压缩的组变量选择

人为分組組結構是针对具有高度相关性的变量而言的。当一组变量具有不可忽略的相关性时，往往在建模时要解决共线性问题

Zou和Hastie（2005）年提出的弹性网（Elastic Net）是最早能解决共线性问题的变量选择方法，其惩罚函数为Lasso惩罚函数和岭惩罚函数的线性组合

$$P_{ENet}(|\beta|; \lambda_1, \lambda_2) = \lambda_1 \sum_{i=1}^p |\beta_i| + \lambda_2 \sum_{i=1}^p \beta_i^2 \quad (11.9)$$

其中 λ_1 、 λ_2 都是调整参数，惩罚函数的第一部分为Lasso惩罚，用于选择变量；第二部分为岭惩罚，能处理高度相关的数据，消除变量间的多重共线性。弹性网的惩罚函数也可以改写为

$$P_{ENet}(|\beta|; \lambda, \alpha) = \lambda \alpha \sum_{i=1}^p |\beta_i| + \frac{\lambda(1-\alpha)}{2} \sum_{i=1}^p \beta_i^2 \quad (11.10)$$

10.5 基于压缩的双层变量选择

Group LASSO的特点是一组变量要么全被选入要么全被剔除，无法在组内选择重要的变量。但是在某些应用中，不仅要选择组，还有在组内选择，例如研究某一疾病发病的影响因素，一个基因由一组变量来描述，但是理论研究表明该组变量中并非每一个都会对该病有显著影响，因此在选择重要基因的同时，还是识别基因中重要的变量。这一过程分为两个层次，第一层选择显著组，第二次选择组内显著单个变量，因此很形象地被称为双层选择（bi-level selection）。

10.5.1 复合型函数双层变量选择

1. *Group Bridge*

Huang等2009年提出的Group Bridge是最早的双层变量选择方法，Group Bridge惩罚函数如下：

$$P_{GBridge}(\beta; \lambda, \gamma) = \sum_{j=1}^J \lambda p_j^\gamma \|\beta^{(j)}\|_1^\gamma \quad (11.11)$$

其中 $0 < \gamma < 1$ ， p_j 是第 j 组变量所包含的个数，在此的作用是权重，即变量数越多权重越大，被压缩的程度越高。

由于Lasso和Bridge都具有单个变量选择的效果，因此Group Bridge具有双层选择功能。

10.5.1 复合型函数双层变量选择

2. *Composite MCP*

Breheny和Huang 在2009年提出的Composite MCP是另一双层变量选择方法，其函数结构与Group Bridge类似，都是复合型函数，但组内和组间惩罚都是MCP函数，惩罚问题为：

$$P_{CMCP}(\beta; \lambda_1, a_1, \lambda_2, a_2) = \sum_{j=1}^J P_{MCP} \left(\sum_{k=1}^{p_j} P_{MCP}(|\beta_k^{(j)}|; \lambda_1, a_1); \lambda_2, a_2 \right) \quad (11.12)$$

类似地，将MCP函数换为SCAD函数时，又有Composite SCAD方法，也能进行双层选择。

10.5.2 稀疏组惩罚

另一类双层选择方法是**稀疏组惩罚** (sparse group penalty), 又称为**可加惩罚** (additive penalty), 它的惩罚函数不是两个具有逐个变量选择功能惩罚的复合函数, 而是构建**逐个变量惩罚**和**仅选择组变量惩罚**的线性组合, 将选择逐个变量和组变量的惩罚函数分开。惩罚函数的一般形式如下

$$P_{indiv}(|\beta|; \lambda_1) + P_{grp}(|\beta|; \lambda_2)$$

其中, $P_{indiv}(\cdot)$ 是具有逐个变量选择功能的惩罚函数, 它作用在每个系数上, 比如Lasso、SCAD和MCP; $P_{grp}(\cdot)$ 是仅具组变量选择功能的惩罚函数, 作用在组变量上, 比如Group Lasso。

10.5.2 稀疏组惩罚

Simon等 (2013) 提出的Sparse Group Lasso (SGL) 属于这类方法, 它通过Lasso和Group Lasso的线性组合来双层选择, 惩罚问题为

$$P_{SGL}(|\beta|; \lambda, \alpha) = \lambda \alpha \|\beta\|_1 + \lambda(1-\alpha) \sum_{j=1}^J \|\beta^{(j)}\|_2$$

式 (11.13) 中的第一项惩罚是Lasso, 用于选择逐个变量, 因此具有变量选择效果的方法如MCP、SCAD等都可以代替它。同样地, 第二项惩罚是Group Lasso, 是为了选择重要的变量组, 理论上任何具有组变量选择功能的方法都能代替该项。SGL方法中, 不同变量的系数或者不同组的系数可能具有同等的重要性, 在进行惩罚时都施以**相同的压缩力度**。但是在实际中应当区别它们的重要性, **不同的系数施加不同的惩罚**, 因此更需要更一般的方法。

10.5.2 稀疏组惩罚

Fang等2014提出的Adaptive SGL满足这一要求，惩罚函数为

$$P_{adSGL}(\|\beta\|; \alpha, \lambda) = (1 - \alpha) \lambda \sum_{j=1}^J w_j \|\beta^{(j)}\|_2 + \alpha \lambda \sum_{j=1}^J \xi^{(j)T} \|\beta^{(j)}\|$$

其中 $w_j \in R_+$ 为第 j 组系数作为一个整体的权重， $\xi^{(j)} = (\xi_1^j, \dots, \xi_{p_j}^j)^T$ 是第 j 组系数内部的权重向量，其元素 $\xi_i^j (i = 1, \dots, p_j)$ 为第 j 组中第 i 个系数 β_i^j 的权重。

因此它对单个系数和组系数分别建立了权重，使不同的组具有不同的重要性，不同的单个系数也具备不完全相同的重要性。当两组权重都为1时，该方法退化为SGL方法。

R语言实现见R代码文件。